Active InferAnt Stream 014.1 ~ Generalized Notation Notation: From Plaintext to Triple Play

Source: YouTube **Playlist**: Active InferAnt Stream **Duration**: 1:45:29 |
Uploaded: Unknown **Transcript method**: auto_caption

All right. Hello, welcome everyone. It's May 15th, 2025. This is active in for ant stream 14.1 and we're going to begin it as we do with a GitHub push and we're going to go from there. Okay, this stream is called generalized notation notation from plain text to triple play. And before jumping into the background, want to give a little bit of the payload up front. Here's the simple structure of the repo and where we're headed. All the source code is in src. All the documentation is in doc and all the output from running main goes to output. So what is the output going to look like? Well, first, where does it start from and where is it going? What it starts from is looking like a markdown file with a controlled vocabulary describing state spaces for a given generative model. And here's where we take that. First, for each of those provided source files, GNN files, we get a bunch of visualizations. So anyone who's worked with PMDP or active inference models knows that figuring out state spaces and doing all kinds of work in transformations with state spaces can be challenging. So one thing this does is it takes that generative model distinction as specified and visualizes it. It outputs it. It exports the model into a variety of formats. JSON, text, XML, graphical, all these different kinds of outports. There's a initial processing report essentially summarizing what kinds of files are being detected. There are rendered simulations. So, here is a PIMDP script that fills in, not perfectly functionally at this point, but I think more than enough to get the picture, a PIMDP script parameterized by the details of the state space of the source file. And where this is all going is to be able to cross render into multiple implementation strategies. So a stub method for going into RX infer type checking is performed. So that is checking the validity of the code and making sure that the source files are valid. And this allows us to bring in not just security features but also something that is often much desired for those running active inference models which is the computational resource estimation. So here we can look at the state space sizes and the different kinds of operations that are entailed for a given generative model and then get different kinds of assessments like what is the relative computational cost going to be

or what is going to be the size for a given number of variables of storing that specific model on on the RAM or on the hard drive. uh LLMs are used at the end to in this case summarize a set of questions. So here are uh reports written with LLM summarizing the state space model. the logs for improved tracking and coding. MCP is the model context protocol and we'll get into this a little bit more but this is a very large file which concatenates all of the MCP methods which we're exposing like visualize GNN file. So this is something that you can run locally or we can stand up and we will stand up MCP servers so that these kinds of services can be provided. Ontology processing these are summaries of the ontology terms used and how they map onto different state spaces. And this is really one of the key pieces of GNN which is putting a little bit of space between putting a flexible interface and gap between a given variable and its ontological annotation. So commonly in looking at these generative models and cognitive modeling type scripts, you'll see the name of a variable like observation modality 1 or latent state or whatever it happens to be some English language name of a variable. However, that can get into some issues because what if it's modality 3, but then one of the other ones comes and goes or what? How do you add modality 4? Or it's the sense uh variable for one agent, but it's the action output from another agent. So in other words, you don't want to have the variable always welded to its onlogical assertion because there can be multiple English terms or roles for a given state space variable. We're going to come back to that with cerebrum. And then also there can be different languages for localization and accessibility. So there's a space between the variable state space and the assertion. And what that looks like inside of the GNN file is a specific section that makes assertions about ontological meaning in that exact generative model. So gone already long long long gone are the days where debating whether the letter B means transition matrix. Well, no. It's just a symbol that in a given generative model might be asserted to mean transition matrix. Then those rendered scripts. So we had the program synthesis of the PIMDP agent parameterized from the GNN file. And then we can actually execute those. And this one hit an error that you'd hit running PIMDP and so did this one. But in previous versions, I've had it running. So it's just in and out all presented as open- source work in progress. and inserting the correct value values into the stem and stub of a PIMDP script is totally doable and it will be coming when the time is right. Then there are test reports. So this is just a test information from a test module and overall a processing summary which has been improved and disimproved and all of that. Okay, so that's where we get with the outputs. How is this all going to be happening? Let's pull back a second. If anyone has any comments or questions, please write them in the

live chat. Um, all this is open source at the GitHub repo active inference institute/generalized notation notation no spaces. I'm using the updated version of cursor 0.50.4 for and the large language model for most of the augmented coding vibe coding happening here is Gemini 2.5 Pro the preview 0506 so just from a few days ago okay we will briefly look at the source structure get a first pass pull back a little bit to the history of GNN then re-enter into the source with people's comments and questions so here's how source folder looks Okay, source folder has a main script. So when you want to replicate the entire process, you just go to src and run python 3 main. So there's a single entry point that is going to look like this. It's going to start go through one, which is just confirming that it has the GNN payload. Go into two, which is the setup phase. That's going to install the requirements, any other packages that you add into requirements.ext. It is going to go into three, which is the test suite. There's just a few tests right now, but you can add more tests. four goes into the type checker suite which has the the type checking as well as the resource estimation methods. After four, it will head into five, the export module. And so the the folders are not numbered, but the scripts are just to really confirm here's the order that we're running these different modules in. Five goes into the export, so you can flag different export modalities. Six into the visualization methods. Seven is the MCP the model context protocol uh section which we'll go into more detail. Eight ontology terms ontology analysis. Nine is the rendering. So that's like the setup of the simulation. That's the transformation from the generic GNN form and parse output into a rendered but not executed EG, PIMDP, RX infer, NGCLERN, etc., etc., etc. We'll build those adapters and they'll be really robust. Step 10 is running the simulation. So, actually this one, let's see if it came through. This is looking like it actually did do the PIMDP simulations. So let us see slow computer. There we go. So not exactly fully functional, but this is demonstrating that we can do program synthesis, controlled vocabulary, program synthesis in an implementation agnostic fashion with static checking, render it into a given target implementation style like PIMDP and then run that PIMDP the method and it looks like there still could be some errors in the simulation but it is breaking on through to the other side. That's 10. Then 11 runs the LLM section which you can customize and it's going to go into this LLM processing and that's where it's going to do things like summarize the uh GNN file or do a set of questions that are responded to by the LLM uh and give an executive summary. Then it's going to finish the 11th one and give you some summaries on the whole pipeline. So again, just while we're overviewing the source folder structure and then heading into the uh context and the the history of GNN main is going to call the 11 scripts and you could just start

up another prompt. Make me a 12th script and a folder for doing dot dot dot. Like if somebody writes a question or a comment in a live chat, I very may well add another module or do some other method. Then each of these scripts which are run by main are going to look into the folder of their own name. So for example, the type checking is going to look into GNN type checker and it's going to find different files like the resource estimator methods all of that. And so you can choose in this sort of main first shell layer of dispatch which tools you want to bring into play. And then we can advance out the tech tree in a really modular way like okay we have type checking on models with planning but we don't have pimdp for it and then we'll just go to the script where it's working and then reverse engineer what is the state space specification we need to do so that we can generate that script that did work and if we can make that connect we're going to be able to have a ton of flexibility and a lot of reproducibility and integrity for the models that we Another feature that's going to come into play in the MCP module is that each module has an MCP. py script. And these are going to be methods that become available to the MCP module. So the MCP module when it is run is going to look in the src folder and say okay go into all the folders and find the MCP. PY and then pull all the methods from the MCP py and that will get reported on in the MCP processing step which is where we get these very very information dense super informative very informationrich files about which methods we're exposing in this implementation. So then if you're running a version and you don't have the execute or you have a certain version of one of these or a different folder, as long as you have that MCP. py, it's going to control which services you're giving model context protocol availability to locally or remotely. And so there's some super fun ways to think about that from like a businessinstitution perspective. Like if it's worth one cent to verify a generative model or to render it into PMDP or to execute it, then maybe with microp payments those kinds of things will become easier to do like X 402 which Holly uh shared in a meeting yesterday. Um, and big picture on the MCP, this is an example of a neuro symbolic architecture because there can be generalist agents, multimodal agents, whether they're probabilistic models like neural networks, transformers, LLMs, etc., or it's a person with a pen and paper and index card. By opening up this GNN stack as an MCP nexus, we're in a new space where the LLM can do program synthesis and execution on generative models. So it can be using its own weights, its own inference methods to do certain things including tool switching. And then if there's a given situation where explicit planning is needed, it's not that we need the LLM or the neural network to do explicit planning, but it could spin up a simulation and dispatch the execution that does do explicit planning. So like if you go to an

LLM and you say, "Hey, what's the expected free energy of me doing this on this day?" It might give you a number or it might give you some relative variable and and that might be directionally accurate. It might be valid. Might be good enough. So it's not to say that it's not. But what it's not doing is doing program synthesis on a generative model and then using free energy minimization to actually work on that model and leaving a full type checked trace all along the way. So that's the sort of neuros symbolic programming aspect that is going to be super interesting. I always think of this like a chocolate chip cookie where the soft cookie part is like the generalist probabilistic agent and then the chocolate chips are like the GNN or the tools that it can call through MCP or through other techniques. So we now have really good dough better and better and really good chocolate fragments better and better too. So there's a lot of opportunities for this kind of crystal embedded in a more liquid or gelatin-like interface that is going to enable models again that don't have explicit planning or other features like that to drop in and deploy these actual simulations with really strong reproducibility and tracing. Okay. So, where did this GNN concept come from? And meanwhile, of course, anyone can write a comment or a question and I'll hope to bring some new features into the package like by the end of the stream. So, this work was during 2022 and 2023 working with Yakobmeckl who's currently a graduate student at Stanford. We were doing various things with multi-agent systems and having a lot of fun talking about active inference and AI and whatnot and we started to explore this idea of making notation systems that were implementation agnostic for active inference and using that for our own epistemic and pragmatic ends. So for example in the stepbystep active inference paper of Smith first and white from 2022 this is like a common step up in terms of learning about generative models. So like in the top left there's D which is a prior and latent states S the latent state itself like the temperature in the room O the observation on the thermometer and then A the mapping between the real temperature in the room and the thermometer. So it's a static onetime step perception. You got a prior on temperature, a thermometer reading coming in and then through the ambiguity matrix A, you come to some posterior on latent state. Then you can add one more motif in with a B transition matrix. Then you can have temperature changing through time emitting observations with thermometer readings. This is a passive inference. It's a partially observable marov process. Then that can get bumped up to a partially observable marov decision process in this case where the decisioning is represented by this pi policy vector of affordances which are operated on by an expected free energy functional which intakes a c preference vector. And then that can even be extended further to have this E habit vector and then this precision temperature

kind of shaky hand on action stuff happening. So it's like okay how do we really look at that though and understand ideally in an open- source plain text way really get down to what are the specific similarities and differences between these models. And so that led to us which we jokingly called LAN gauge because of gauge theory. Um where these are plain text and you can take a given column from like the KOD and you can just paste it into a notepad and it will come out like a markdown and it has different sections and it's a pretty similar structure today two years later. Looking in the GNN examples, looking here, we see some of those similar fields and a similar state space notation as to what it was where you declare the state space, you declare the connections, give initial parameterization, add any other equations, how is time treated, what is the annotation of the ontology. Um, and then we could have other sections like what's a footer or we could have a cryptographic signature. So that was version one of the paper in the earlier days of active inference institute fun where we wanted to highlight that we could bring together all these different ways of thinking about program synthesis and we could express it in what is also the title of the stream which is the triple play. So the triple play is the highfidelity rendering of a GNN expression whether it's a small motif or whether it's a whole ecosystem scale multi-agent generative model a hybrid synthetic intelligence etc etc etc across and here's the triple play plain text graphical and executable computational forms moving in and out of these forms and that's the triple So then we talk a little bit about those three aspects of it. The the textbased model, plain text, benefits of plain text, graphical models here of course playing on the double play of graphical itself being like a graphical relational model but also being visual, diagrammatical, graphical and then the executable cognitive models. So sometimes you just want to like dream a big dream about a recipe, but there's recipes you can specify that you can't cook at home. You just don't have the ingredients or no kitchen could possibly do this or that. So separating out where are we just writing creative speculative semi-fiction generative model play? When are we visualizing the aforementioned? And then when do we really want to execute it? And when it gets down to executing it, just like software can be implemented on different operating systems, we want to be able for flexibility and reproducibility to execute cognitive models in different settings. So that was our paper. uh we summarized that shortly and then we provided a quick assessment on how this GNN is actually not just useful and pragmatic, it's also very epistemic. So we dive a little bit into the realism and instrumentalism topic and then we talk about how we will be able to implement this triple play. And here we are in that ballpark, the very same implementing the triple play and even planting some seeds that bloomed many many April

showers and May flowers later. Like from a linguistics perspective, it would be interesting to explore possible grammatical case systems, morphology, and dialects associated with GNN. It's not just a throwaway line. That's what cerebrum is about. case systems for generative models. So now we can say well we want the locative version of that GNN. And so for those who are wondering where does cerebrum come into play? How can I use what I've learned from Sanskrit and Russian and Latin and is invisible in English? How can I use that kind of morphological morphos syntactic understanding of language? How can I bring that to high reproducibility generative modeling? Here is how. And that was the paper in 2023 with Yakob and put it on the back burner for a little bit. But now the repo is back with a uh Fury, but not an Angry One. And hopefully this is going to be a really key tool and platform for people who want to use generative models for education and for research and it can be under the hood. It's not that every single person may be in this exact repo. But as we start to think through these things as a community, it becomes a lot clearer and clearer how having these methods for setting up workspaces, testing our environment, type checking and resource estimation, exporting and confirmation, visualizing generative models, making methods available for LLMs and other agents, understanding the onlogical annotations of state spaces, preparing ing simulations through rendering executing the simulations and using LLMs to give interpretability in natural language and do other kinds of operations like these are some of the core activities that we need for our own research work. So it's cool that we can develop this and make this an open science space um and welcome participation from from people who see opportunities and challenges here. So let's do a few. Let's check out the agenda and see where we might want to do something. And then again, if somebody's watching live, they can write a comment or an idea in the chat. So what is GNN and why is it important? GNN is a plain text notational style and tradition that focuses on statesbased definition of generative models. For example, statistical models or cognitive models of agents. Why it's important is that with that kind of a controlled structural vocabulary for generative models, it's possible to reproducibly and at high throughput engage with a whole sequence of different kinds of functionalities ranging from reproducible environment setup and testing, typeing, resource estimation, exporting, visualization, accessibility to MCP, ontological analysis, rendering, execution, LMS, and beyond and more. So that's pretty cool and important. Core concepts are woven throughout and the goals are for us as always on these kinds of streams and at the institute where everyone's welcome to find out more and participate is to increase the accessibility, rigor, and applicability of active inference. And so this is a triple play on that triple play. How

does GNN allow for clear textual specification of complex models? So, we can look at what it looks like now, but with more advanced automated code generation, for example, the autogen lib may be one uh library to look into as well as kit. So, kit is a code synthesis library. So right now this is like a V 0.1 or a V1 version of the GNN spec. It isn't that it has to be written this way with this section title. It's just to give an idea and then we can use more advanced code templating methods to generate these from possibly even a more distilled condensed version that renders into a GNN that then gets used. But that's kind of like the assembly code question. Um, so how does GNN allow for clear textual specification of complex models? Well, by separating out a few key steps. Separating out first in the state space block the sheer declaration of the size and type of different variables. So C is going to be length three vector float. Or you could say, well, I'm I'm going to be able to do my C vector with integers. Then you could say it's an int. And then in the resource and checking, you could know that you could work with integer calculations. So that's the state space block. Then connections, which is a little bit generic. Of course, these need to be nuanced a lot to be really used as a base graph, but the information is there after the state space block and the structural specification is the initial parameterization. So here it could be done in a more pythonic fashion or it can be just fully specified out what is the initial state or a given kind of renderer could have a default like if something is not provided with initial state it's defaulted to uniform or to the identity matrix. So whatever kind of rendering rules you want and then you separate out the ontology annotation. So you've already basically initialized the space of the model and initialized the values and now you have this annotation layer which could easily be swapped out with a different language. Okay. So that's kind of a a 2023 brought into current year plain text version of what a minimal GNN looks like which is just a focus on some free form contextual information about the generative model state space for initialization with initial parameterization and the ontology annotation. And we've been having some great convos in the multi- aent meetings about well what space does that leave open then for the orchestration and the message passing and the communication all these kinds of logistics and operations of generative models. But that's where actually again the cerebrum and the case system comes into play because if the generative model let's just say is the ingredients like that we have um lentils, mustard and vinegar. And then there's a sentence in the recipe which is like the orchestration specification like with the mustard in the kitchen put the sauce on the lentils and then being able to look at all those prepositions and understand which of the cases are each of the models being used in. Oh, it's with the mustard but it's on the lentils. But then if lentil has

multiple methods or multiple different characteristics and it turns out like with the lentils is really hard on the GPUs because they're heavy but putting something on the lentils doesn't mean that we need GPUs at all. So that's where we get case specific resource estimation through this kind of metal linguistic intelligence layer key syntax and structural elements just went through them. Okay, Triple Play with GNN. And and no, I I'm not just talking about the everpresent comprehensive who's on first style dialogue between Professor Phineas Cogwell and Pipsqueaky Wheeler as they discuss the GNN format. I'm not just talking about that triple play. I'm talking about the representation triple play which the LLM invented another triple play of understanding conversion execution but it works well too so Levy okay showcasing GN in action practical examples of the triple play so let us do that and also see a little bit about how cursor does this kind of lowkey high throughput high efficacy program synthesis so let's make a new example and work within the framework we already have and then we can explore some more far out things. So write a new example GNN file fully compliant like these two for a pom dp where the ontology assertions are in Spanish terms. There are more submatrices for each variable, but still compatible with just just these are just kind of reminder notes. But in in a in a better world, which we may already be in, these don't even matter, but still compatible with visualization methods. And the situation is that it is a trading algorithm being run on an airplane where if it loses money then the plane runs out of gas. Okay. So with all this kind of documentation and context and especially templates and examples in the niche then Gemini or whomever you call will be able to make a new example. though let us see we're seeing into that that chain of thought that inner dialogue that Gemini is having with itself and it's starting to think through okay so here's what we have to do this kind of pseudo thought that's happening and then it's writing out all these details so it'll pop up a file soon okay meanwhile so welcome Andrew I'm going to copy in your question. So now now we're in the game. Let's just get a few uh interesting things done during the rest of the stream. Okay. Andrew wrote, "Could a 12 involve putting together some dashboard of the visualizations and/or metric reporting or some synthesis between LLM stage and visualization? Sure. Let's do let's make one new folder called site. This this I'm sure could be done automatedly too, but I'm just going to manually make an MCV. py uh mainly bring in the init. Um then okay, stop thinking. Continue. Most used command, right? Okay, started up a new chat agent in cursor over here. And so I'm going to drag in main readme and site and say write a file in src. Again, just dragging it in for explicit reference. for 12 site. py which will call on all the methods in site folder make them all comprehensively for making a website single HTML file with

sections which reflects the output folder, eg embedding all the visualizations and LLM outputs and etc. Okay, we're still deep in the thought chain. Okay, we're we're in the air we're in the Spanish-sp speakaking airplane trading palm DP example. So, it's kind of funny uh and I guess apt, but for those who have looked at the 2022 active inference textbook, this is kind of like the chapter 6 approach. And it's also the same thing that for years we've been working on in terms of just like what is the recipe for cognitive modeling like wow it sounds really cool about doing all this active inference work but how do we really go from there being a situation of interest and then whether for whimsy and fun or just delight or something more professional like systems engineering or doing something like an analyst would. How do you really go from just a system of interest to having a cognitive model in hand? And chapter 6 from the 2022 textbook took that part of the way and it gave like a sort of proto recipe, but this is bringing it into another level of structural possibilities. And not to constrain either, more like a playground where there's all kinds of fun jumps and pull-ups and swings you can do because your expressivity is higher with these kinds of structures and tools if you know what's up. So, airplane trading on the palm DP. So as per requested we have the Spanish language ontological assertions but the whole point is it doesn't change anything with what the uh state space block or any of the initial parameterization is. So let's accept that. I'm going to clear the terminal, delete the output folder just to be sure and then rerun Python 3 main. So in an ideal setting which we do not know yet if we are in or not we will have a seamless welllogged step through of the new GNN trading airplane trading palm DP it should just work. So we'll we'll see if we get there but bigger picture we know that we can get there. We're going to see will it make the visualizations that we want. So, here comes this is okay. Here's the airplane trading. Yeah. So, now we have the the uh we have the variables from the airplane with their Spanish onlogical assertions being rendered. So it's like single prompt. You don't need to know about the details. You can go in there and you can do the engineering and set the specific state spaces. Oh no, no, no, no. We wanted it to be six, not three or whatever it happens to be. But again, big picture, we can render simulations for all of these generative models to our heart's delight. So will this script run? Well, I mean, again, we can sort of find out. Did it run? What happens? Yeah, not sure if the run script worked perfectly, but again, it's just a question of okay, what model class is rendered what way and then expanding that out. So that's one key feature to demonstrate which is just purely within cursor let alone using more more sophisticated methods. We can do oneshot program synthesis or really like meta notation synthesis and use that to just play into the same well-typed pathways so that we can make a Spanish localized uh

simulation about doing trading on an airplane right off the bat. So that might take a while if you were going to manually write all those state spaces. So, I'll just drop that one into archive. Okay. So, okay. So, that was a example of making an making a new generative model. So we are again like update your priors if you're not here and seeing this. We have single shot chat mediated capacity to write certain structures of generative model and then there's a lot of work to be done so that increasingly general styles of GNN can be parsed and rendered and so on. But that's actually why the the whole suite is structured as it is because I suspect that if we confirm which version of GNN we're working in, set up our workspace, run the tests, do the static checking, export and confirm that the export was successful, visualize it, and then execute and uh large language model it, we're going to be in a great spot. Okay. So, Tatia Zot, I'm going to read comment. You wrote, "My best friend. Thank you. Can you prepare group Discord page your YouTube channel? I want to join you." Yes. Please go to discord.activeference.institute um and you'll you'll find the discord link. So for those who are in the live chat now, I'm just I just went to welcome.activeinference.institute. That's like the landing page if you want to get involved. We do have a discord and you're totally welcome to to join in and I'll paste the link just directly into the YouTube live chat as well. Um and that's one place activity happens, but it happens wherever you are. There you are. So here we go with the uh demonstrating a sort of package level flexibility in terms of being able to bring in a new functionality into the GNN space. So here we have the site folder coming together and then it will make a 12 site. py and then it will add that into main so that it runs 12 site so that when we add a new example in and we run it all with main that will have it all pop up. So let's do another example. So here I've created src site MCP. So this is like a way to confirm yeah these are the methods related to the website like generate pipeline summary site that's going to be exposed through MCP. It's not fully functional yet, but it's all there. So that uh people could call sequential MCP calls and then say like here's the recipe. You blend up A, then you add B, take two of C and combine them with D, then do all this and do that and then that. So that level of uh programmatic synthesis is something that is very excitingly on offer from DSPI which is a programming structure. It'll look a little cleaner in the repo for doing program synthesis at the level of LLMs, which is where we're going to have the MCP at. So, DSPI is going to be useful for programming sequences of MCP type calls. kit or something similar is going to allow program synthesis and codebase level intelligence. MCP whether it's fast MCP or another method is going to support making these tools available even if somebody doesn't have the whole repo because we can offer an endpoint. Um as earlier mentioned cerebrum and

this sort of lexical intelligence paradigm is going to help us make case specific renderings of generative model. So that when we understand from the beginning what roles a given object or generative model is going to serve, we're going to be able to understand what kinds of rendering pipeline are needed. So again, like if we're talking about a table, are we talking about placing something on the table or are we talking about hitting a nail with the table? Because if we're just going to place something on the table, we only need enough RAM to pick up the object that we're putting on the table. Like we're going to put the pen on the table. But if we're going to put the table on the pen, then we need something like a GPU that can pick up a table. So that's how prepositions are and what they signal. And then there's just again these these folders can can have more documents and more different frameworks brought in. X42 which we were talking about yesterday is just thinking okay how could we do microp payments so that if somebody thinks I'm going to ring up the institute and ask for a textbook written in this language and then we do that on the back end or I'm going to have a generative model written up like this then there can be small charges or just kind of authentication different kinds of methods to to offer these up as as services. Okay, the pipeline 12 is integrated. So let's see if the main can handle this running. So just clear I'm going to delete the output folder. That's why the output folder is standalone because how satisfying to have SRC and doc. That's it. And this is like the minimal dual channel strategy. Basically, doc is suggestive that it's going to be more natural language like src is suggestive that it's going to host all of the code and it is that way. And then we have a few top level scripts. So we have the citation if uh anyone sees relevance in the work partially filled. I'll change the release date to today for version 0.1.0. Code of conduct auto written but we can tweak that too. contributions already. Even those just listening or watching or reviewing summaries thereof, it's people and agents like you that make GNN such a great tool. It's so true. License MIT license for now. Those with more insight into how we can have more advanced and multi- channelannel licenses get in touch, etc. But that's where it's at for now. And as far as I understand, it's the first program synthesisbased generative model creation assistance tooling framework. So I hope that people who find value in that can get in touch, maybe support the institute, maybe support the project, maybe use the framework. The readme is a overview summary. We have a cyber physical and cognitive security policy in this GNN project. So inquire within for more and we do want to try to be supportive. So whether it's during this live chat and you have questions you want to ask or whether you have GitHub issues that you want to open up or other questions about like what can we do with this, how can I etc. then we're there for

support. But these are kind of the top level files and all of the operational stuff is in source. all of the extraneous documentation as well as uh different other ideas like how could this be useful for multi-agent simulations like well maybe we expand the GNN structure to include an agent block and then we have a GNN file that is the notation for a multi-agent simulation and it says you're going to use one instance of you know five instances of this GN GNN and one instance of that GNN and so we can make structured notational fields for doing all that kind of simulation level reproducibility and communication so that and then we can uh have either kind of uh state spaces that are wired up again separate from their English name. So we can flip in and out of using that state space representation where the variables could be known like by their cryptographic hash uh all the way on through just okay now you want to turn it into this language lens. So just ideas about how we could use GNN and structured fields for the the communication questions and keeping with this spirit and inspiration that it's implementation agnostic. GNN isn't the rendered version that's executable. GNN is the notation notation generalization. So if a given kind of simulation level or agent level description is written in GNN like DNA like source code then you may have a rendering element that is able to render and transform the contents into the format you want. That's docs. Okay. Pipeline has finished. It says uh successful 11 one success with a warning. So let's see what the site. Okay, looks like 12 thinks that the site was created. So let's find out. Okay, here's the summary site. Moment of truth. wouldn't say it's the uh most advanced site that I've ever seen, but it does the job. It has all the information and then that's how you develop modules. just like okay now develop site to be like this so here I'll just remind it of that ensure the readme includes site and the 12th script make any improvements within the site folder ensuring you use separate scripts within site if it is getting like long modular methods. Clear. So it will take some uh some work some collaboration from hopefully all to many of us that will make it more rigorous. For example, having a strict template of a certain kind like hey, if you want to do discretet time, discrete state spaces and get it to land the plane in PIMDP, here is the standard example or here's the code template, the code generation template, the kind of stuff we can do with uh code generation libraries. Here's the code generation library format schema for if you want to make sure that it's going to go into PIMDP. Well, like this should be like this and you need this, you can't have that, etc. Um, let's add a little functionality with RX infer. So while the website stuff is happening, assess and use at web search to understand how RX infer at model statements are used. Eg the generative model joint distributions in format where it can be parsed by RX infer into a executable model. Then make those improvements to so that here's the rendering.

So that rendered outputs are accurately conveying all the information make and RX infer dot or utils py or rx infer utils.jl if needed. Yeah, the Gemini 2.5 sometimes stalls out kind of randomly. Uh we'll we'll just say, "All right, I'm going to make this edit." And then just hangs there. Uh but generally speaking, it has been great. Okay, let's add another example. See if we can get into some other kind of model that we uh Okay, so we have website chugging away here. RX infer improvement here. Now we're going to write another example. According to these compliant examples, write another kind of generative model, not a POM DP that will creatively and functionally demonstrate important Features and capacities of [Music] package. Make the situation something esoteric yet tangible. Semicolon. available though ineffable. Make it best best best possible. Um, it's been 1 hour. If some super awesome funny stuff happens, I might stay on. But otherwise, just a few more minutes because I think I have pretty much conveyed where it's at and some of the core features. So, in the last section here, like if anyone has an idea for an example or another function that they want to see or they have a specific question that they would like addressed in this stream, please write it in the next few minutes. Okay. Greetings, Andreas. Oh, now that could be a fun a fun plot twist to include some of your work into this. Okay, I'll make a just a sort of starter pack NTQR and a uh NTQR or GNN NTQR. R MD just leave that there for now, but I'm I'm sure that you and others could uh imagine a few different ways that that NCQR comes into play. Uh it could be like a module offered in a multi- aent simulation setting like just as another module you could run on top or after the multi- aent then you can run it with this NTQR layer. Just trying to delete this but it's a slow computer. Okay. Okay. So, this is what the uh the esoteric Oh, goodbye cursor. It was good to know you. Pull quit. Okay, we're back. Okay. All right. So, it's a GNN basian network that stochastically generates a verse concept. It aims to model the flow of poetic inspiration from emotional and elemental inputs to concrete poetic characteristics. So, pretty cool. This is getting us into the uh space of of explicit active inference language models. So not just using the inputs and outputs of language models and treating those or conditioning those with active inference but this is actually getting into what we can now do. Now make another example absolutely comprehensive and vast called an active inference language model that has state spaces for eg sound ponym syntax morpho syntax semantics situational narrative, all the nested and interacting levels so we can really truly actually finally seriously safely and beneficially have a full GNN based model of language. No training data sets here. Just oneshotting the triple play based upon the GNN launchpad for making the world's first actual active inference linguistic intelligence agent toolbox. box. Okay. Have it

continue with RX infer cuz I got quit on and looks like the website one did too. We'll just say continue and then we'll we'll run it again in a few seconds. Yeah, also the new version of cursor 050. It has some background. Well, one important thing indexing was broken out into its own separate tab. If you've watched a bunch of in for ant streams, you know that I was always going to like features and then scrolling down to the codebase and and doing the reync. Um, there's MCP, but this is exactly how easy it's going to be to then have all this GNN stuff tucked away and then point the MCP server at it. So then other people or agents will be using cursor or whatever and then it'll be like plan make a plan and then that could invoke this program synthesis rendering type checking visualization simulation execution multiple environments logging etc etc etc website generation um and also there's the uh background agents I don't know where it is in the settings. I haven't looked at it, but there's a background agent that you can have. So that could be very useful. For example, a background agent could say always absolutely vigilantly be assessing if everything is totally good and up to mega standards for everything we're doing. And just let us know if not, but don't make changes by yourself. Okay. AI LM phonetic target and articulation. Let's see what this looks like. So again, if this were using motifs and the structure that we knew was going to work, then like we would know it would work. Okay, let's close that chat. See if the website is done. All right, here we go. We're going to throw caution to the wind and see if this active inference language model will run in PIMDP. It's It's not going to. Well, let let's delete the output just to confirm. Delete the output folder. Okay, so let's just see how it broke it up. see how the LLM broke up this structured analysis. And this is just a total total early hint of the power of using language models and general purpose agents to write structured texts which we then can bring into this explicit program execution framework. Okay, so hidden state factors, phonetic target and articulation. Then there's a lexical morphos syntactic level. So lexical concept ID that would be like the diction like table chair. Then there's the current syntactic rule. This takes us right back to cerebrum and the different grammatical cases like is the table acting? Is the table the recipient of action? Is something being placed on the table? Is something being placed near of the table? And then there's tense, number, gender, etc. That brings us from the sort of lexical into the more properly fragments of semantics which here let's see semantic proposition. So a propositional availenced propositional assertionbased semantic representation not the only one but that's why we can make multiple kinds of GNN files. We could say make a new GNN file that explores a really really different and radically contrasting semantic concept narrative context internal goals predictions observation modalities you can make no no we only want it

to observe auditory we want to have confirmation that the observations are only coming in through sound and that all the semantics is being endogenously generated or we we want the policy output to be red, yellow, green, just three states or we want the policy output to be words in the dictionary then we would have literally next token prediction policy prediction in this kind of ailm would be just like it is in transformers. So this state space block is obviously massive. Um the connections. Let's see if the visualization went through. Okay, here's the poetic muse model. Interesting. Here's the poetry model. Ontology assertions in the state space. That one was oneshotted. And then let's look at the language model. Okay, looks like it has some differences that um so that we could get to this state space representation, but that the but like it it could be a breaking change. But the kind of uh suggestion would be drag this in. Hey, we didn't get all the visualizations that we thought of. Ensure that visualizations are generated properly for this and then it would be off to the races again. Exports. Here's its attempt to export it in JSON style. So maybe you only need to use maybe you're when you're calling MCP or you're you're using this for your own research, maybe all you want to do is specify it, type check it, and then export it. You don't even want to visualize it. Or maybe you you just you don't care if it's type checked, you just want to visualize it. So that's why these methods are are written out separately. Uh this is the language model failing to be put into PIMDP format. Let's see if it gets a little bit better. And similarly RX infer uh didn't continue through comprehensively ensure that at model blocks really are output to there given the GNN input. So maybe I'll run one more. Okay. Okay. Refactor the website. All right. So we we'll let this uh RX infer update come through. But we could in the LLM when I just interrupted it. That would be So here we get that LLM um interpretation generated from an OpenAI call at this point to describe the model and to to answer different questions about the model. So you can have a default question kit q kit ask these different questions. Um let's see what happened when it was trying to execute it. Yep. A matrix must be a numpy array. At least it's a real error. Um, so where to go is going to depend on where you're at, what time it is, all that kind of stuff. I hope that people look through, contribute, improve. I think one of the most high yield payoffs if somebody is uh familiar with structured code generation methods or type checking um or program synthesis techniques would be to develop a uh template. not not to fix it as like the one and only template to use, but just here's a template that we can reference all of the other methods against and and uh also know which degrees of freedom we're going to be able to to uh move with and break because right now it maybe we'll say oh well um you know the visualization wanted there to be a a hyphen here and then you add the hyphen here and the

visualization works but then the other you know something else doesn't work. So that is where I believe that using this order of operations is going to be helpful or something similar to it of course pending everyone's input but basically like first verify that the GNN module is loaded up and use this file structure punctuation to confirm you're like yeah this is the version of GNN we're working with this is what the symbols mean then but that that should just output the documentation and the version. It shouldn't need any requirements to be installed in the workspace. Then setup also it's kind of standalone and simple but it it should install the requirements that are needed. then an extremely comprehensive test suite can be run including tests about later modules. So maybe there's more maybe there's a module that's not used in your flow but still it is type uh checked and tested with a template. The type checking is basically the first of the real ingress steps and this is where I know that there are better methods that that people can bring to the table. Okay, let's let's run main one more time just to see if it can get this RX infer plane landed. Okay, so GNM type checker uh maybe the resource estimator could be broken out to its own script or or folder even. But the resource estimation is not costly. So I figured it probably can stay with the type checking because it is based upon the inferences made by the static type checker engine export that that could go slightly earlier uh because you don't need to type check but type checking tells you yeah this is you're going to have an issue with with this variable it's not stated properly or like you won't and then export kind of confirms that that you can read in and out. So if everything is green light up on through export, the goal is like you should be in the clear because we've already done static testing. We've already demonstrated that we can read in and then read out a given file. So like downstream of five, I think it it may even use the re-exported form after having already been checked to be the same as the input. So it's like if we can regenerate the original GNN file 100%. then we may want to work with that airgapped version purely because on one hand it's like it'd be the same but on the other hand it's a version that we know that we wrote visualization there there could be generic and specific visualization methods and so that would require a lot of flagging like where are we doing heat maps where are we doing other kinds of visualizations MCP again that that could be moved to to a different uh order but it should expose these methods and the neurosymbolic possibilities for MCP use like imagine the so-called language models the transformer LLM calling the real language model as program synthesis and then that active inference probabilistic language model could have a GNN dialect. So it could do program synthesis on GNN within a linguistic GNN representation inside of a cookie of just probabilistic general inference ontology. That one could just be more

of an archival uh latter note. It doesn't have to be eight, but it's pretty interesting. Like if we wanted to say, hey, this undergraduate course or this microcertification or training, you are going to fully understand this drone forest simulation example. It's a multi- aent simulation with drones in a forest. Here's the file that we are going to be looking at and everything is going to be about those ontology terms. So, every single thing we say is going to be semantically laser eye on your education related to drones in the forest. And we can have one foot in that content and still pivot out and associate and talk about other topics, but let's be clear with which ontology terms we're using in a controlled fashion. render maybe another word is slightly preferable but this is just preparing the simulation for execution. So having a render like a prepare and then the execution phase may just be as simple provably as Python and then the Python script name. That would be ideal. Doesn't do any other stuff. It just pulls the trigger on what render loaded up. LLM again here you can just say make it so that LLM does this other kind of thing. That's just more general LLM tools and then today on the stream we made this site to generate it. So let's uh in the closing bit. Okay, let us look at the RX infer to see if if we got a better RX infer simulation. Okay, that one didn't. That one didn't. Okay, so program synthesis for RX infer waits till another day. One thing that's going to be awesome with RX infur specifically is another degree of freedom where we will be able to specify RX infer models and then we could either run that locally or dispatch that to the RX and FUR server API. So then that's like earlier I said you can write recipes that you can't cook but this is like separating out now it's like you can write a recipe that you know you can order because you can fully specify it and then send it to the to another endpoint that's just an RX and first server that will run your app model with your conditions but you've already done the type checking and the verification. you know that you're working off of a generic implementation agnostic representation. Okay. And the LLM is still um processing. So let us uh let that finish then pull in the site. But in terms of the last um content here, we will uh read a little bit from the plain text to triple play. So, we're going to improve this with one prompt and then see if anybody uh has any comments or questions after reading it. We'll write a new who's on first dialogue. Triple play. Write a new novel dialogue. Who's on first style? Whimsical and deadly serious. Total Americana. Deep lore. Absolutely conveying all the topics with quote triple play puns related to dragging the dock folder in such that through the reading of such the SRC code becomes strangely obvious to those who pay attention. Again, this must be absurd speculative realism for a variety of reasons. delivered in PhD dissertation length who's on first dialogue style or it could be three conversants if that will make the triple play pun play all the Thank

you, Andrew P. Okay. We could have like a we could do a benchmark on how much do each of these LLMs know about 1920s baseball Okay, while that chain of thought proceeds, let's just see the LLM stuff. So, it's written four of the five. So each each um currently in the GNN folder examples is the target input directory. We might want to have another directory called input or something so that it's just obvious which files are going to be uh iterated through. But obviously that's one of the um ways to work faster. If you only have one uh file in here, like it's going to happen way faster. But right now, it has to do all the iterating and looping for all five of these examples. So I think I'll move I'll move three of them into archive just so that people can get started and get it running faster. But the language they all could be developed in versions in really fun ways. Okay. We'll generate this dialogue and then if there's any uh last comments to see what else is in the docs folder about GNN. This just summarizes that's a starting place. Here's a domain specific language DSL spec sheet for it. A uh more human legible uh examples breaking down the different examples. file structure these kinds of markdown representations uh mermaid plots super useful. How does it get implemented? Again the goal and the whole vision is like specifying generative models at a level above implementation. So if if you're on a Intel versus an AMD processor, the program is still going to work. So similarly, like if you're going to be exe, there might be some features that are easier or only available within one of these computer languages or within one of these implementation environments. But in principle, we're talking about specifying something at at a more semantic level. uh as mentioned and alluded to a little bit, I think that this kind of executable program synthesis is going to have a lot of relevance for neurosymbolic synthetic hybrid architectures. There's a whole world of multi-agent specification and ways that we can work at this layer. Gemini, one more chance really fully write out this conversation which we need. Uh here's the plain text or a summary of it of the paper itself, but it's linked in the in the repo with Yakob. Some information on the syntax, but this could be reflected probably better using this type of kit concept. Uh, no. This is the repo. Paste /kit. Codebased mapping, symbol extraction, code search, building LLM powered, developer tools, agents, and workflow. When I saw that, I was like, yes. and making MCP servers access to it. Uh that's basically the the docs just to review what's there. Now X42 microservices RX infer we can make a PIMDP folder just to be leaving a stub there. And that could have like more generic documentation information on doing GNN PIMDP, but it it'll just be to be. All right. Here we go. All right. I'm going to push it on GitHub. read this uh oneshotted dialogue. Oh, the LLM is still working for some reason. Okay. uh read the dialogue and then either it will uh either it will end on an

extremely dramatic funny note we haven't looked at it yet or maybe somebody will have an appropriate comment or question in the next minute or two at which point I will then read that one. So here we are in other triple play dialogue. Okay, here we go. The GNN triple play A dialogue in three fits and many errors characters Abner double day earnest precise a true believer in the system capitalized. He's trying to explain the rules of generalized notation notation GNN. He often clutches a heavily annotated copy of what looks like a rule book but is in fact doc/GNN file structure doc. Casey Specs Stumble, an old-time baseball manager, weathered, full of Cracker Barrel philosophy and prone to spectacular misunderstandings. He's always looking for the angle or the specs on the new talent and how this all plays out on the diamond, which he sometimes confuses with the GNN connections graph. He squints at everything through thick smudged spectacles. branch. Ricky Wright, a shadowy, almost spectral figure. Perhaps a long disenfranchised league commissioner, a rogue statistician, oh brother. Or a philosopher of the great American pastime, now haunting the onlogical back alleys of model specification. He speaks in aracular, often unsettling pronouncements, his voice like the creek of an old stadium gate. setting the deepest, dustiest back office of the National Generative Model League, NGML headquarters. A forgotten annex that smells of old leather, stale cigars, lenammen, and the faint sharp tang of ozone from ancient calculating machines that were rumored to predict not just game outcomes, but the very fabric of fan belief. Yellowed schematics of factoraphs and basets are pinned to the peeling wallpaper, interspersed with faded dgeray types of stern-faced men in bowler hats who might have been pioneering statisticians, spiritualists, or perhaps both. A single fly specked gas lamp sputters erratically, casting long dancing shadows that make the GNN diagrams on the wall seem to shift and writhe. A ticker tape machine in the corner occasionally coughs out cryptic decontextualized messages. Eg. model name not found, state space undefined, policy does not converge. The curtain rises, or rather the dust mo swirl expectantly. Abner is pacing, occasionally tapping his rulebook. Specs is trying to whittle a piece of what might be a broken bat or a very large pencil with a pen knife. Rickety is half hidden in the shadows, only a glint of an eye and the occasional wisp of pipe smoke visible. Bit the first. Who's our model name? Abner clearing his throat, adjusting his spectacles. Gentlemen, welcome. Or rather, welcome back to the eternal deliberation. Today we must rigorously define the parameters of our our understanding. It all begins as it must with the model name. Clarity here is paramount. Specs. Model name. Eh, is he a new kid? What's he about? Lefty, righty? Got any pop? We need pop. The fans, they scream for pop. And where's he from? Don't tell me he's another one of them

fancy college boys who can't hit a curveball in the state space block. Abner size. A small puff of dust rises from his wa waist coat. No specs. model name isn't a player. It's it's the designation, the unique identifier for the entire conceptual architecture we are attempting to instantiate. Think of it as the banner under which the whole enterprise marches, the grand title on the spine of the almanac. Select the team name, the generative giants, the notation nationals, the markoff chain marauders. I had a team once, the Mudville 9. Good bunch of fellas. Terrible fielders. Lost 17 straight in the time horizon. 17. Each discreet time step of fresh agony. Rickety voice rasps from the shadows like dry leaves skittering across an abandoned home plate. The name. Yes. Before the word there is the naming. Before the connections are drawn, the model name is whispered into the void. It is the first casting of the die. the initial prior block from which all subsequent beliefs unfold. Does the name sing to the act infotology annotation or does it merely echo in the empty grandstands of failed hypothesis? Precisely, Mr. Wright. Well, almost. It's less about singing and more about unambiguous referencing as stipulated in doc/file structure doc. MD section 2. It's a descriptor, a label. For example, GNN PMDP agent. Concise, informative. PMDP agent sounds foreign. Is he one of them international league fellas? Got a good arm. This agent, can he play multiple positions? Like, can he cover hidden state and also fill in observation if we're in a pinch? Versatility. That's the ticket in this league. A real triple play threat if you catch my drift. Abner massaging his temples. It's PMDP refers to the Python library for marov decision processes. The agent is the model itself. It's not about positions on a field specs but about variables in a system. Variables. Eh. So like guys who are inconsistent, hot one day, cold the next. We got a whole roster full of variables. S_F0 couldn't hit water if he fell out of a boat last Tuesday. And Ocore M1, don't get me started on Ocore M1's fielding errors. Cost us the penant that one. Initial parameterization was all wrong for him. Rickety, the initial parameterization, the primordial soup, the conditions at the dawn of the simulation. Was the bag cked from the start? Were the dice loaded in the state space block? Or was it merely a d_f0 set to uniform ignorance awaiting the harsh lessons of observation? Yes, the d_f0 vector the prior belief over initial states for the factor zero. You see specs model name sets the stage. Then comes the model Model annotation. Is that the fellow who writes down what model name does? Like a scorekeeper or one of them newspaper scribblers always looking for a model name to tear down. We had a guy Scoop Mcury used to say my policy vector was more like a meandering suggestion. The nerve Abner the model annotation is a free text caption specs. It explains the model's purpose, its context. It's ras. It's the color commentary if you will to the model names

playbyplay. It's where you describe the triple play of GNN, the textbased specification, the graphical model intuition, and the executable cognitive model. Triple play. Now you're talking, Abner. So model name is the batter. Model annotation is the guy on first. And who's on second? Is it State Space Block? He sounds like a big fella, a slugger. This is going to be a long time horizon. State Space Block isn't a who. It's a what. It's the where. It's the definition of all possible states the system can be in. The dimensions of a diamond itself. I'm going to skip on since there's many sections to the end, but let's skip. Okay, last section. Abner looks from specs to rickety a dawning realization or perhaps just fatigue in his eyes. So, src/main.py py that orchestrates the one GNN parsing, the two setup environment configuration, the three export transformations, the four GNN type checker validation, and the five render execution. That's just the Empire calling the game, ensuring the rules of generalized notation notation are followed. The Empire, of course, that's main. always sticking his nose in, checking the GNN version and flags, and making sure the footer and the signature are in order. Thinks he's God or G or some other fancy letter, but without him, it's all just output/log/kaos.txt. Rickety, the main py, the silent orchestrator, the loom upon which the GNN tapestry is woven. It does not judge the model annotation's poetry nor the equation's elegance. It merely ensures the pipeline flows from text to graph to code. A humble servant of the triple play. The true mystery, gentlemen, is not who's on first or what's on second or even I don't know on third. It's why we play the game at all. And for that answer, you'll need to consult a model name far grander than any found in SRC/GNN/amples. Rickety fades further into the shadows. Spec starts idly checking model parameters on an imaginary bat. Abner stares at his GNN rule book. A single bead of sweat tracing a path through the dust on his forehead. The ticker tape machine clatters one last time. Simulation complete. Observed outcome unlikely under current priors suggest revising world models. Or perhaps the rules of the game itself. The gas lamp sputters and dies, plunging the room into near darkness, illuminated only by the faint residual glow of unvalidated hypothesis. All right, thanks everyone. Hope this was pragmatic and epistemic for you. Looking forward to people's feedback and collaborations. Acting first surf.